Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи №6

“ Автоматизоване тестування Python-програм ”

Виконала:

Студентка групи ФЕП-33с

Най Анастасія

Перевірив:

Проф. Свелеба С.А.

Львів-2026

Мета роботи: сформувати практичні навички професійного автоматизованого тестування Python-програм із використанням фреймворку pytest, навчитися проєктувати unit та integration tests, створювати fixtures, використовувати параметризацію, перевіряти exceptions, ізолювати файлову систему та зовнішні залежності, застосовувати mocks і аналізувати test coverage.

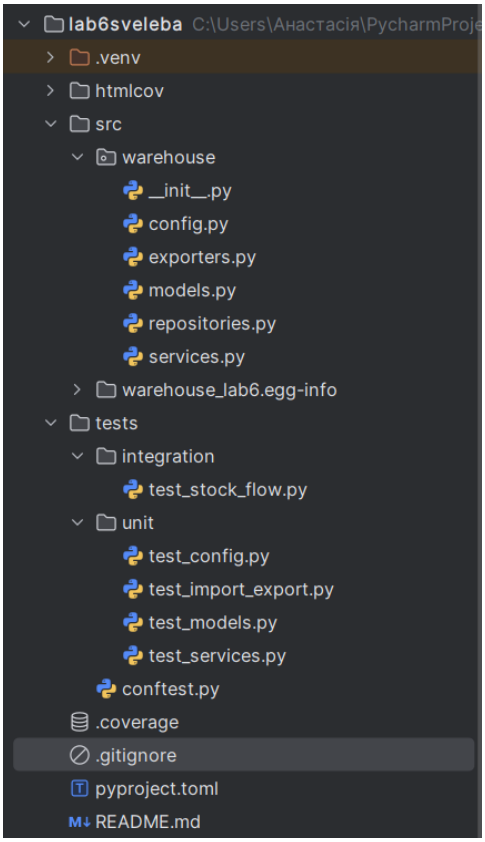
Варіант 10. Система обліку складських запасів

Протестувати:

- створення Product/WarehouseItem та валідацію quantity/price;
- операцію надходження;
- операцію списання;
- insufficient stock;
- пошук за кодом товару;
- розрахунок загальної вартості складу;
- визначення low-stock positions;
- boundary та invalid quantity/unit_price;
- CSV streaming import;
- malformed quantity record;
- JSON export;
- configuration low-stock threshold;
- WarehouseRepository через Mock/fake;
- supplier/reorder service через Mock із side_effect;
- async reorder provider через AsyncMock та integration stock flow.

Хід роботи:

1. Спочатку я створила робочу папку проєкту. Організувала професійну структуру за стандартом src-layout: у папці src/warehouse/ зосередила весь програмний код, залишивши корінь проєкту для конфігураційних файлів (pyproject.toml, README.md) та каталогу тестів tests/ із розділенням на unit/ та integration/.



2. У файлі src/warehouse/__init__.py я налаштувала пакет warehouse для організації модулів системи керування складськими запасами та забезпечила можливість імпорту необхідних компонентів пакета.

```
"""Warehouse stock management package."""

from src.warehouse.config import get_low_stock_threshold
from src.warehouse.exporters import (
    WarehouseValidationError,
    export_json,
    stream_csv_import,
)
from src.warehouse.models import (
    InsufficientStockError,
    Product,
    WarehouseItem,
)
from src.warehouse.repositories import InMemoryWarehouseRepository
from src.warehouse.services import WarehouseService

__all__ = [
    "Product",
    "WarehouseItem",
    "InsufficientStockError",
    "WarehouseValidationError",
    "InMemoryWarehouseRepository",
    "WarehouseService",
    "get_low_stock_threshold",
    "stream_csv_import",
    "export_json",
]
```

3. У файлі pyproject.toml я налаштувала конфігурацію Python-проєкту, параметри запуску pytest, автоматичне виявлення тестів, підтримку

асинхронного тестування та вимоги до покриття коду тестами за допомогою Coverage..

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "warehouse-lab6"
version = "1.0.0"
description = "Warehouse stock management and pytest testing"
requires-python = ">=3.10"
dependencies = []


[project.optional-dependencies]
test = [
    "pytest>=8.0",
    "pytest-cov>=5.0",
    "pytest-asyncio>=0.23",
]


[tool.setuptools.packages.find]
where = ["src"]

[tool.pytest.ini_options]
asyncio_mode = "auto"
testpaths = ["tests"]
pythonpath = ["src"]
addopts = "-ra --strict-markers"
markers = [
    "integration: integration tests for warehouse operations",
]

[tool.coverage.run]
branch = true
source = ["src/warehouse"]
relative_files = true

[tool.coverage.report]
show_missing = true
skip_covered = false
fail_under = 80
```

4. У файлі src/warehouse/models.py я розробила основні сутності бізнес-логіки Product і WarehouseItem із валідацією даних товару та кількості, реалізувала методи надходження (receive) і списання (issue) товарів, властивість обчислення загальної вартості складської позиції, а також виняток InsufficientStockError для обробки ситуацій недостатньої кількості товару на складі.

```
"""Domain models and validation for warehouse stock."""

from dataclasses import dataclass
import math


class InsufficientStockError(Exception):
```

```python
    """Raised when an issue operation exceeds available stock."""


def _validate_quantity(quantity: int) -> None:
    """Quantity must be a non-negative integer, excluding bool."""
    if isinstance(quantity, bool) or not isinstance(quantity, int):
        raise ValueError("Quantity must be an integer.")

    if quantity < 0:
        raise ValueError("Quantity cannot be negative.")


def _validate_positive_quantity(quantity: int) -> None:
    """Operations must move at least one item."""
    if isinstance(quantity, bool) or not isinstance(quantity, int):
        raise ValueError("Quantity must be an integer.")

    if quantity <= 0:
        raise ValueError("Quantity must be greater than zero.")


@dataclass(frozen=True)
class Product:
    """A product identified by a unique stock code."""

    code: str
    name: str
    unit_price: float

    def __post_init__(self) -> None:
        if not isinstance(self.code, str) or not self.code.strip():
            raise ValueError("Product code cannot be empty.")

        if not isinstance(self.name, str) or not self.name.strip():
            raise ValueError("Product name cannot be empty.")

        if isinstance(self.unit_price, bool):
            raise ValueError("Unit price must be a number.")

        if not isinstance(self.unit_price, (int, float)):
            raise ValueError("Unit price must be a number.")

        if not math.isfinite(self.unit_price):
            raise ValueError("Unit price must be finite.")

        if self.unit_price < 0:
            raise ValueError("Unit price cannot be negative.")

        # Normalize integer prices to float.
        object.__setattr__(self, "unit_price", float(self.unit_price))


@dataclass
class WarehouseItem:
    """A product and its current quantity in the warehouse."""

    product: Product
    quantity: int = 0

    def __post_init__(self) -> None:
        if not isinstance(self.product, Product):
            raise TypeError("product must be a Product instance.")

        _validate_quantity(self.quantity)

    @property
```

```
    def stock_value(self) -> float:
        """Return the total monetary value of this stock position."""
        return self.product.unit_price * self.quantity

    def receive(self, quantity: int) -> None:
        """Add stock to the warehouse."""
        _validate_positive_quantity(quantity)
        self.quantity += quantity

    def issue(self, quantity: int) -> None:
        """Remove stock or raise an exception if stock is insufficient."""
        _validate_positive_quantity(quantity)

        if quantity > self.quantity:
            raise InsufficientStockError(
                f"Cannot issue {quantity} units of "
                f"{self.product.code}; available: {self.quantity}."
            )

        self.quantity -= quantity
```

5. У файлі src/warehouse/config.py я реалізувала конфігурацію системи з можливістю налаштування порогу низького залишку товарів через змінну середовища WAREHOUSE_LOW_STOCK_THRESHOLD, передбачивши перевірку коректності заданого значення.

```
"""Configuration helpers for the warehouse application."""

import os


DEFAULT_LOW_STOCK_THRESHOLD = 5


def get_low_stock_threshold() -> int:
    """Read the low-stock threshold from the environment."""
    raw_value = os.getenv(
        "WAREHOUSE_LOW_STOCK_THRESHOLD",
        str(DEFAULT_LOW_STOCK_THRESHOLD),
    )

    try:
        threshold = int(raw_value)
    except (TypeError, ValueError) as exc:
        raise ValueError(
            "WAREHOUSE_LOW_STOCK_THRESHOLD must be an integer."
        ) from exc

    if threshold < 0:
        raise ValueError("Low-stock threshold cannot be negative.")

    return threshold
```

6. У файлі src/warehouse/repositories.py я визначила інтерфейс репозиторію для роботи зі складськими даними та реалізувала репозиторій у пам’яті для додавання, пошуку й отримання всіх складських позицій, включно з перевіркою унікальності кодів товарів.

```
"""Repository abstractions and an in-memory implementation."""
```

```python
from typing import Protocol

from src.warehouse.models import Product, WarehouseItem


class WarehouseRepository(Protocol):
    """Interface expected by the warehouse service."""

    def add(self, item: WarehouseItem) -> None:
        ...

    def get(self, product_code: str) -> WarehouseItem | None:
        ...

    def all(self) -> list[WarehouseItem]:
        ...


class InMemoryWarehouseRepository:
    """Store warehouse items in memory, indexed by product code."""

    def __init__(self) -> None:
        self._items: dict[str, WarehouseItem] = {}

    def add(self, item: WarehouseItem) -> None:
        if not isinstance(item, WarehouseItem):
            raise TypeError("item must be a WarehouseItem instance.")

        code = item.product.code

        if code in self._items:
            raise ValueError(f"Product {code} already exists.")

        self._items[code] = item

    def get(self, product_code: str) -> WarehouseItem | None:
        return self._items.get(product_code)

    def all(self) -> list[WarehouseItem]:
        return list(self._items.values())

    def register_product(
        self,
        product: Product,
        quantity: int = 0,
    ) -> WarehouseItem:
        """Create and store a new warehouse position."""
        item = WarehouseItem(product=product, quantity=quantity)
        self.add(item)
        return item
```

7. У файлі src/warehouse/services.py я реалізувала сервісний рівень керування складськими запасами, який забезпечує реєстрацію та пошук товарів, надходження і списання продукції, обчислення загальної вартості запасів, визначення товарів із низьким залишком і асинхронне оформлення замовлень постачальнику.

```python
"""Warehouse business logic and reorder integration."""

from typing import Protocol

from src.warehouse.config import get_low_stock_threshold
```

```
from src.warehouse.models import Product, WarehouseItem
from src.warehouse.repositories import WarehouseRepository


class ReorderProvider(Protocol):
    """Interface for an external supplier/reorder service."""

    async def reorder(self, product_code: str, quantity: int) -> bool:
        ...


class WarehouseService:
    """Coordinate warehouse operations using a repository."""

    def __init__(
        self,
        repository: WarehouseRepository,
        low_stock_threshold: int | None = None,
    ) -> None:
        if low_stock_threshold is None:
            low_stock_threshold = get_low_stock_threshold()

        if (
            isinstance(low_stock_threshold, bool)
            or not isinstance(low_stock_threshold, int)
            or low_stock_threshold < 0
        ):
            raise ValueError(
                "Low-stock threshold must be a non-negative integer."
            )

        self.repository = repository
        self.low_stock_threshold = low_stock_threshold

    def register_product(
        self,
        product: Product,
        quantity: int = 0,
    ) -> WarehouseItem:
        """Register a product and its initial stock."""
        item = WarehouseItem(product=product, quantity=quantity)
        self.repository.add(item)
        return item

    def find_product(self, product_code: str) -> WarehouseItem | None:
        """Find a stock position by product code."""
        return self.repository.get(product code)

    def receive(self, product_code: str, quantity: int) -> WarehouseItem:
        """Receive stock for an existing product."""
        item = self.repository.get(product_code)

        if item is None:
            raise KeyError(f"Product {product_code} was not found.")

        item.receive(quantity)
        return item

    def issue(self, product_code: str, quantity: int) -> WarehouseItem:
        """Issue stock for an existing product."""
        item = self.repository.get(product_code)

        if item is None:
            raise KeyError(f"Product {product_code} was not found.")

        item.issue(quantity)
```

```
        return item

    def total_value(self) -> float:
        """Return the combined value of all warehouse stock."""
        return sum(item.stock_value for item in self.repository.all())

    def low_stock_items(self) -> list[WarehouseItem]:
        """Return positions at or below the configured threshold."""
        return [
            item
            for item in self.repository.all()
            if item.quantity <= self.low_stock_threshold
        ]

    async def reorder_low_stock(
        self,
        provider: ReorderProvider,
        reorder_quantity: int = 10,
    ) -> dict[str, bool]:
        """Ask a provider to reorder every low-stock product."""
        if (
            isinstance(reorder_quantity, bool)
            or not isinstance(reorder_quantity, int)
            or reorder_quantity <= 0
        ):
            raise ValueError("Reorder quantity must be a positive integer.")

        results: dict[str, bool] = {}

        for item in self.low_stock_items():
            results[item.product.code] = await provider.reorder(
                item.product.code,
                reorder_quantity,
            )

        return results
```

8. У файлі src/warehouse/exporters.py я реалізувала імпорт товарних даних із CSV-файлів із потоковою обробкою записів і перевіркою коректності вхідних даних, а також експорт складських даних у формат JSON. Для обробки некоректних даних передбачила спеціальний виняток WarehouseValidationError.

```
"""CSV import and JSON export utilities."""

import csv
import json
from collections.abc import Iterable, Iterator
from pathlib import Path
from typing import TextIO

from src.warehouse.models import Product, WarehouseItem


class WarehouseValidationError(ValueError):
    """Raised when an imported warehouse record is malformed."""


def stream_csv_import(
    source: str | Path | TextIO,
) -> Iterator[WarehouseItem]:
    """
```

```
    Stream warehouse positions from CSV.

    Expected header:
        code,name,quantity,unit_price
    """
    should_close = False

    if isinstance(source, (str, Path)):
        handle = open(source, "r", encoding="utf-8-sig", newline="")
        should_close = True
    else:
        handle = source

    try:
        reader = csv.DictReader(handle)
        expected_fields = {"code", "name", "quantity", "unit_price"}

        if reader.fieldnames is None:
            raise WarehouseValidationError("CSV file is empty.")

        if not expected_fields.issubset(set(reader.fieldnames)):
            raise WarehouseValidationError(
                "CSV must contain code,name,quantity,unit_price columns."
            )

        for line_number, row in enumerate(reader, start=2):
            try:
                code = row["code"]
                name = row["name"]
                quantity_text = row["quantity"]
                price_text = row["unit_price"]

                if None in (code, name, quantity_text, price_text):
                    raise ValueError("Missing field.")

                product = Product(
                    code=code.strip(),
                    name=name.strip(),
                    unit_price=float(price_text),
                )
                quantity = int(quantity_text)

                if quantity < 0:
                    raise ValueError("Quantity cannot be negative.")

                yield WarehouseItem(product=product, quantity=quantity)

            except (TypeError, ValueError, KeyError) as exc:
                raise WarehouseValidationError(
                    f"Invalid CSV record at line {line_number}: {exc}"
                ) from exc
    finally:
        if should_close:
            handle.close()


def export_json(
    items: Iterable[WarehouseItem],
    destination: str | Path | TextIO,
) -> None:
    """Write warehouse stock positions as JSON."""
    payload = [
        {
            "code": item.product.code,
            "name": item.product.name,
            "quantity": item.quantity,
```

```
            "unit_price": item.product.unit_price,
            "stock_value": item.stock_value,
        }
        for item in items
    ]

    if hasattr(destination, "write"):
        json.dump(payload, destination, ensure_ascii=False, indent=2)
        destination.write("\n")
        return

    path = Path(destination)
    with path.open("w", encoding="utf-8") as handle:
        json.dump(payload, handle, ensure_ascii=False, indent=2)
        handle.write("\n")
```

9. У файлі tests/conftest.py я створила спільні тестові фікстури для підготовки об’єктів товарів, складських позицій, репозиторію та сервісу, що дозволяє повторно використовувати тестові дані в різних тестових модулях.

```
"""Shared pytest fixtures."""

import pytest

from src.warehouse.models import Product, WarehouseItem
from src.warehouse.repositories import InMemoryWarehouseRepository
from src.warehouse.services import WarehouseService


@pytest.fixture
def product() -> Product:
    return Product(
        code="P001",
        name="Keyboard",
        unit_price=1250.50,
    )


@pytest.fixture
def warehouse_item(product: Product) -> WarehouseItem:
    return WarehouseItem(product=product, quantity=10)


@pytest.fixture
def repository() -> InMemoryWarehouseRepository:
    return InMemoryWarehouseRepository()


@pytest.fixture
def warehouse_service(
    repository: InMemoryWarehouseRepository,
) -> WarehouseService:
    service = WarehouseService(
        repository=repository,
        low_stock_threshold=5,
    )

    service.register_product(
        Product(code="P001", name="Keyboard", unit_price=1250.50),
        quantity=10,
    )
    service.register_product(
```

```
        Product(code="P002", name="Mouse", unit_price=500.00),
        quantity=3,
    )

    return service
```

10. У файлі tests/unit/test_models.py я розробила модульні тести для перевірки створення товарів і складських позицій, валідації даних, обчислення вартості запасів, надходження та списання товарів, а також обробки помилок у разі некоректної кількості або недостатнього складського залишку.

```
"""Unit tests for Product and WarehouseItem."""

import math

import pytest

from src.warehouse.models import (
    InsufficientStockError,
    Product,
    WarehouseItem,
)


def test_product_creation(product):
    assert product.code == "P001"
    assert product.name == "Keyboard"
    assert product.unit_price == pytest.approx(1250.50)


def test_product_accepts_zero_price():
    product = Product("FREE", "Promotional item", 0)
    assert product.unit_price == 0.0


@pytest.mark.parametrize("code", ["", " ", "\t"])
def test_product_rejects_empty_code(code):
    with pytest.raises(ValueError, match="code"):
        Product(code, "Keyboard", 100)


@pytest.mark.parametrize("name", ["", " ", "\n"])
def test_product_rejects_empty_name(name):
    with pytest.raises(ValueError, match="name"):
        Product("P001", name, 100)


@pytest.mark.parametrize("price", [-1, -0.01, float("-inf"), float("inf"),
math.nan])
def test_product_rejects_invalid_price(price):
    with pytest.raises(ValueError):
        Product("P001", "Keyboard", price)


@pytest.mark.parametrize("price", ["100", None, True])
def test_product_rejects_non_numeric_price(price):
    with pytest.raises(ValueError, match="number"):
        Product("P001", "Keyboard", price)


def test_warehouse_item_creation(warehouse_item):
```

```
    assert warehouse_item.quantity == 10
    assert warehouse_item.product.code == "P001"


@pytest.mark.parametrize("quantity", [-1, -10])
def test_warehouse_item_rejects_negative_quantity(product, quantity):
    with pytest.raises(ValueError, match="negative"):
        WarehouseItem(product, quantity)


@pytest.mark.parametrize("quantity", [1.5, "5", True, None])
def test_warehouse_item_rejects_non_integer_quantity(product, quantity):
    with pytest.raises(ValueError, match="integer"):
        WarehouseItem(product, quantity)


def test_stock_value_uses_quantity_and_unit_price(warehouse_item):
    assert warehouse_item.stock_value == pytest.approx(12505.0)


def test_receive_increases_quantity(warehouse_item):
    warehouse_item.receive(5)
    assert warehouse_item.quantity == 15


def test_issue_decreases_quantity(warehouse_item):
    warehouse_item.issue(4)
    assert warehouse_item.quantity == 6


def test_issue_exact_available_quantity(warehouse_item):
    warehouse_item.issue(10)
    assert warehouse_item.quantity == 0


def test_issue_more_than_available_raises(warehouse_item):
    with pytest.raises(InsufficientStockError, match="available"):
        warehouse_item.issue(11)

    # A failed issue must not modify stock.
    assert warehouse_item.quantity == 10


@pytest.mark.parametrize("quantity", [0, -1, -5])
def test_receive_rejects_non_positive_quantity(warehouse_item, quantity):
    with pytest.raises(ValueError, match="greater than zero"):
        warehouse item.receive(quantity)


@pytest.mark.parametrize("quantity", [0, -1, 1.2, True, "2"])
def test_issue_rejects_invalid_quantity(warehouse_item, quantity):
    with pytest.raises(ValueError):
        warehouse_item.issue(quantity)
```

11. У файлі tests/unit/test_services.py я реалізувала модульні тести сервісного рівня для перевірки роботи репозиторію та сервісу керування складом, використовуючи заглушки й mock-об’єкти для перевірки взаємодії між компонентами, обробки помилок і асинхронного оформлення замовлень постачальнику.

```
"""Unit tests for warehouse services and repository."""
```

```python
from unittest.mock import AsyncMock, Mock

import pytest

from src.warehouse.models import (
    InsufficientStockError,
    Product,
    WarehouseItem,
)
from src.warehouse.repositories import InMemoryWarehouseRepository
from src.warehouse.services import WarehouseService


def test_repository_add_and_get(product):
    repository = InMemoryWarehouseRepository()
    item = WarehouseItem(product, 4)

    repository.add(item)

    assert repository.get("P001") is item
    assert repository.get("UNKNOWN") is None


def test_repository_rejects_duplicate_code(product):
    repository = InMemoryWarehouseRepository()
    repository.add(WarehouseItem(product, 1))

    with pytest.raises(ValueError, match="already exists"):
        repository.add(WarehouseItem(product, 2))


def test_repository_all_returns_all_items():
    repository = InMemoryWarehouseRepository()
    first = WarehouseItem(Product("A", "First", 10), 1)
    second = WarehouseItem(Product("B", "Second", 20), 2)

    repository.add(first)
    repository.add(second)

    assert repository.all() == [first, second]


def test_register_product(warehouse_service):
    item = warehouse_service.register_product(
        Product("P003", "Monitor", 7000),
        quantity=2,
    )

    assert warehouse_service.find_product("P003") is item
    assert item.quantity == 2


def test_find_product_returns_none_for_unknown_code(warehouse_service):
    assert warehouse_service.find_product("UNKNOWN") is None


def test_receive_updates_stock(warehouse_service):
    item = warehouse_service.receive("P001", 5)
    assert item.quantity == 15


def test_issue_updates_stock(warehouse_service):
    item = warehouse_service.issue("P001", 4)
    assert item.quantity == 6
```

```python
def test_receive_unknown_product_raises(warehouse_service):
    with pytest.raises(KeyError, match="not found"):
        warehouse_service.receive("UNKNOWN", 1)


def test_issue_unknown_product_raises(warehouse_service):
    with pytest.raises(KeyError, match="not found"):
        warehouse_service.issue("UNKNOWN", 1)


def test_service_issue_insufficient_stock(warehouse_service):
    with pytest.raises(InsufficientStockError):
        warehouse_service.issue("P002", 4)

    assert warehouse_service.find_product("P002").quantity == 3


def test_total_warehouse_value(warehouse_service):
    # P001: 10 * 1250.50; P002: 3 * 500.
    assert warehouse_service.total_value() == pytest.approx(14005.0)


def test_low_stock_uses_configured_threshold(warehouse_service):
    low_stock = warehouse_service.low_stock_items()

    assert [item.product.code for item in low_stock] == ["P002"]


def test_repository_mock_return_value():
    repository = Mock()
    expected_item = WarehouseItem(Product("X", "Test", 10), 2)
    repository.get.return_value = expected_item

    service = WarehouseService(repository, low_stock_threshold=5)

    assert service.find_product("X") is expected_item
    repository.get.assert_called_once_with("X")


def test_repository_mock_side_effect():
    repository = Mock()
    repository.get.side_effect = [
        WarehouseItem(Product("A", "First", 10), 1),
        None,
    ]
    service = WarehouseService(repository, low_stock_threshold=5)

    assert service.find_product("A") is not None
    assert service.find_product("B") is None
    assert repository.get.call_count == 2


def test_service_interacts_with_repository():
    repository = Mock()
    repository.get.return_value = WarehouseItem(
        Product("A", "Test", 10),
        5,
    )
    service = WarehouseService(repository, low_stock_threshold=5)

    service.receive("A", 2)

    repository.get.assert_called_once_with("A")
    assert repository.get.return_value.quantity == 7
```

```python
def test_invalid_low_stock_threshold_is_rejected():
    with pytest.raises(ValueError, match="threshold"):
        WarehouseService(Mock(), low_stock_threshold=-1)


@pytest.mark.asyncio
async def test_reorder_low_stock_uses_async_mock(warehouse_service):
    provider = Mock()
    provider.reorder = AsyncMock(return_value=True)

    result = await warehouse_service.reorder_low_stock(
        provider,
        reorder_quantity=10,
    )

    assert result == {"P002": True}
    provider.reorder.assert_awaited_once_with("P002", 10)


@pytest.mark.asyncio
async def test_reorder_side_effect_reports_supplier_failure(
    warehouse_service,
):
    provider = Mock()
    provider.reorder = AsyncMock(side_effect=[False])

    result = await warehouse_service.reorder_low_stock(
        provider,
        reorder_quantity=8,
    )

    assert result == {"P002": False}
    provider.reorder.assert_awaited_once_with("P002", 8)


@pytest.mark.asyncio
async def test_reorder_rejects_invalid_quantity(warehouse_service):
    provider = Mock()
    provider.reorder = AsyncMock()

    with pytest.raises(ValueError, match="positive"):
        await warehouse_service.reorder_low_stock(provider, 0)

    provider.reorder.assert_not_awaited()
```

12. У файлі tests/unit/test_import_export.py я розробила тести для перевірки імпорту даних із CSV-файлів і експорту в JSON, зокрема обробки потоків і файлів, перевірки обов’язкових стовпців, виявлення некоректних значень та обробки порожніх файлів.

```python
"""Unit tests for CSV import and JSON export."""

import io
import json

import pytest

from src.warehouse.exporters import (
    WarehouseValidationError,
    export_json,
    stream_csv_import,
)
```

```
from src.warehouse.models import Product, WarehouseItem


def test_csv_import_from_file_object():
    source = io.StringIO(
        "code,name,quantity,unit_price\n"
        "P001,Keyboard,10,1250.5\n"
        "P002,Mouse,3,500\n"
    )

    items = list(stream_csv_import(source))

    assert len(items) == 2
    assert items[0].product.code == "P001"
    assert items[0].quantity == 10
    assert items[0].product.unit_price == pytest.approx(1250.5)
    assert items[1].product.name == "Mouse"


def test_csv_import_streams_records():
    source = io.StringIO(
        "code,name,quantity,unit_price\n"
        "P001,Keyboard,10,1250.5\n"
    )

    result = stream_csv_import(source)

    # The importer returns an iterator rather than a complete list.
    assert iter(result) is result
    assert next(result).product.code == "P001"


def test_csv_import_from_path(tmp_path):
    csv_file = tmp_path / "stock.csv"
    csv_file.write_text(
        "code,name,quantity,unit_price\n"
        "P001,Keyboard,7,100\n",
        encoding="utf-8",
    )

    items = list(stream_csv_import(csv_file))

    assert len(items) == 1
    assert items[0].quantity == 7


def test csv import rejects missing columns():
    source = io.StringIO("code,name\nP001,Keyboard\n")

    with pytest.raises(WarehouseValidationError, match="must contain"):
        list(stream_csv_import(source))


def test_csv_import_rejects_invalid_quantity():
    source = io.StringIO(
        "code,name,quantity,unit_price\n"
        "P001,Keyboard,not-a-number,100\n"
    )

    with pytest.raises(WarehouseValidationError, match="line 2"):
        list(stream_csv_import(source))


def test_csv_import_rejects_negative_quantity():
    source = io.StringIO(
        "code,name,quantity,unit_price\n"
```

```
        "P001,Keyboard,-1,100\n"
    )

    with pytest.raises(WarehouseValidationError, match="line 2"):
        list(stream_csv_import(source))


def test_csv_import_rejects_invalid_price():
    source = io.StringIO(
        "code,name,quantity,unit_price\n"
        "P001,Keyboard,5,invalid\n"
    )

    with pytest.raises(WarehouseValidationError, match="line 2"):
        list(stream_csv_import(source))


def test_csv_import_rejects_empty_file():
    source = io.StringIO("")

    with pytest.raises(WarehouseValidationError, match="empty"):
        list(stream_csv_import(source))


def test_json_export_to_path(tmp_path):
    items = [
        WarehouseItem(Product("P001", "Keyboard", 100), 3),
        WarehouseItem(Product("P002", "Mouse", 50), 2),
    ]
    output_file = tmp_path / "stock.json"

    export_json(items, output_file)

    data = json.loads(output_file.read_text(encoding="utf-8"))

    assert len(data) == 2
    assert data[0]["code"] == "P001"
    assert data[0]["quantity"] == 3
    assert data[0]["stock_value"] == pytest.approx(300.0)


def test_json_export_to_file_object():
    items = [WarehouseItem(Product("P001", "Keyboard", 100), 2)]
    destination = io.StringIO()

    export_json(items, destination)

    data = json.loads(destination.getvalue())

    assert data[0]["name"] == "Keyboard"
    assert data[0]["stock_value"] == pytest.approx(200.0)


def test_json_export_empty_list(tmp_path):
    output_file = tmp_path / "empty.json"

    export_json([], output_file)

    assert json.loads(output_file.read_text(encoding="utf-8")) == []
```

13. У файлі tests/unit/test_config.py я створила тести для перевірки конфігурації порогу низького залишку товарів, зокрема використання значення за замовчуванням, читання змінної середовища та обробки некоректних і від’ємних значень.

```python
"""Unit tests for environment-based configuration."""

import pytest

from src.warehouse.config import (
    DEFAULT_LOW_STOCK_THRESHOLD,
    get_low_stock_threshold,
)


def test_default_low_stock_threshold(monkeypatch):
    monkeypatch.delenv("WAREHOUSE_LOW_STOCK_THRESHOLD", raising=False)

    assert get_low_stock_threshold() == DEFAULT_LOW_STOCK_THRESHOLD


def test_custom_low_stock_threshold(monkeypatch):
    monkeypatch.setenv("WAREHOUSE_LOW_STOCK_THRESHOLD", "12")

    assert get_low_stock_threshold() == 12


def test_invalid_low_stock_threshold(monkeypatch):
    monkeypatch.setenv("WAREHOUSE_LOW_STOCK_THRESHOLD", "abc")

    with pytest.raises(ValueError, match="must be an integer"):
        get_low_stock_threshold()


def test_negative_low_stock_threshold(monkeypatch):
    monkeypatch.setenv("WAREHOUSE_LOW_STOCK_THRESHOLD", "-1")

    with pytest.raises(ValueError, match="cannot be negative"):
        get_low_stock_threshold()
```

14. У файлі tests/integration/test_stock_flow.py я реалізувала інтеграційні тести для перевірки повного циклу роботи складської системи: реєстрації товарів, надходження та списання продукції, імпорту даних із CSV, експорту в JSON, виявлення низьких залишків і виконання відповідних операцій із поповнення запасів.

```python
"""Integration tests for complete warehouse workflows."""

import io
import json
from unittest.mock import AsyncMock, Mock

import pytest

from src.warehouse.exporters import export_json, stream_csv_import
from src.warehouse.models import Product
from src.warehouse.repositories import InMemoryWarehouseRepository
from src.warehouse.services import WarehouseService


@pytest.mark.integration
def test_complete_stock_flow():
    repository = InMemoryWarehouseRepository()
    service = WarehouseService(repository, low_stock_threshold=3)

    service.register_product(
```

```
        Product("P001", "Keyboard", 100),
        quantity=10,
    )

    service.receive("P001", 5)
    service.issue("P001", 4)

    item = service.find_product("P001")

    assert item is not None
    assert item.quantity == 11
    assert service.total_value() == pytest.approx(1100.0)


@pytest.mark.integration
def test_csv_import_to_warehouse_and_json_export(tmp_path):
    csv_source = io.StringIO(
        "code,name,quantity,unit_price\n"
        "P001,Keyboard,5,100\n"
        "P002,Mouse,2,50\n"
    )

    repository = InMemoryWarehouseRepository()

    for item in stream_csv_import(csv_source):
        repository.add(item)

    service = WarehouseService(repository, low_stock_threshold=2)
    output_file = tmp_path / "warehouse.json"

    export_json(repository.all(), output_file)

    exported = json.loads(output_file.read_text(encoding="utf-8"))

    assert service.find_product("P001").quantity == 5
    assert service.find_product("P002").quantity == 2
    assert service.total_value() == pytest.approx(600.0)
    assert [row["code"] for row in exported] == ["P001", "P002"]


@pytest.mark.integration
@pytest.mark.asyncio
async def test_low_stock_reorder_workflow():
    repository = InMemoryWarehouseRepository()
    service = WarehouseService(repository, low_stock_threshold=3)

    service.register product(
        Product("P001", "Keyboard", 100),
        quantity=10,
    )
    service.register_product(
        Product("P002", "Mouse", 50),
        quantity=2,
    )

    provider = Mock()
    provider.reorder = AsyncMock(return_value=True)

    result = await service.reorder_low_stock(provider, reorder_quantity=10)

    assert result == {"P002": True}
    provider.reorder.assert_awaited_once_with("P002", 10)

    # Reordering requests stock from the supplier; it does not
    # automatically change the on-hand quantity.
    assert service.find_product("P002").quantity == 2
```

15. У файлі README.md я підготувала документацію проєкту з описом призначення системи керування складськими запасами, її структури, основних можливостей і команд для встановлення залежностей та запуску тестів.

```
# Лабораторна робота №6
## Автоматизоване тестування Python-програм за допомогою pytest

**Варіант №10:** керування складськими запасами.

### Мета роботи

Ознайомитися з автоматизованим тестуванням Python-програм за допомогою
pytest, створити модульні та інтеграційні тести, перевірити нормальні,
граничні й помилкові сценарії та оцінити покриття коду.

### Функціональність

- створення товарів і складських позицій;
- перевірка ціни та кількості;
- надходження та видача товарів;
- обробка недостатнього залишку;
- пошук за кодом товару;
- розрахунок загальної вартості запасів;
- пошук товарів із низьким залишком;
- налаштування порога низького залишку;
- потоковий імпорт CSV;
- експорт даних у JSON;
- тестування залежностей через Mock і AsyncMock.

### Встановлення

Створи та активуй віртуальне середовище, потім виконай:

```bash
python -m pip install -e ".[test]"
```

Запуск усіх тестів

```bash
python -m pytest -v
```

Запуск із покриттям коду та гілок

```bash
python -m pytest -v --cov=warehouse --cov-branch --cov-report=term-missing --
cov-report=html
```

Результати покриття

HTML-звіт створюється в каталозі `htmlcov/`.
Відкрий `htmlcov/index.html` у браузері.

Мінімальний поріг покриття, встановлений у конфігурації: 80%.
Фактичний результат визначається після запуску тестів.
```

16. У файлі .gitignore я визначила перелік файлів і каталогів, які не потрібно додавати до репозиторію, зокрема віртуальне середовище, кеш Python, тимчасові файли pytest і результати формування звітів про покриття коду.

```
# Python
__pycache__/
*.py[cod]

# Virtual environments
.venv/
venv/

# Pytest
.pytest_cache/

# Coverage reports
.coverage
.coverage.*
htmlcov/

# IDE
.idea/
.vscode/
```

Результати:

Клонування / Створення середовища:

```
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab6sveleba> python -m venv .venv
Unable to copy 'C:\\Program Files\\WindowsApps\\PythonSoftwareFoundation.Python.3.13_3.13.3824.0_x64__qbz5n2kf
сія\\PycharmProjects\\lab6sveleba\\.venv\\Scripts\\python.exe'
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab6sveleba> .venv\Scripts\activate
```

Встановлення залежностей проєкту:

```
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab6sveleba> python -m pip install -e ".[test]"
Obtaining file:///C:/Users/%D0%90%D0%BD%D0%B0%D1%81%D1%82%D0%B0%D1%81%D1%96%D1%8F/PycharmProjects/lab6sveleba
  Installing build dependencies ... done
  Checking if build backend supports build_editable ... done
  Getting requirements to build editable ... done
  Preparing editable metadata (pyproject.toml) ... done
Collecting pytest>=8.0 (from warehouse-lab6==1.0.0)
  Using cached pytest-9.1.1-py3-none-any.whl.metadata (7.6 kB)
Collecting pytest-cov>=5.0 (from warehouse-lab6==1.0.0)
  Using cached pytest_cov-7.1.0-py3-none-any.whl.metadata (32 kB)
Collecting pytest-asyncio>=0.23 (from warehouse-lab6==1.0.0)
  Using cached pytest_asyncio-1.4.0-py3-none-any.whl.metadata (4.1 kB)
Collecting colorama>=0.4 (from pytest>=8.0->warehouse-lab6==1.0.0)
  Using cached colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Collecting iniconfig>=1.0.1 (from pytest>=8.0->warehouse-lab6==1.0.0)
  Using cached iniconfig-2.3.1-py3-none-any.whl.metadata (2.8 kB)
Collecting packaging>=22 (from pytest>=8.0->warehouse-lab6==1.0.0)
  Using cached packaging-26.3-py3-none-any.whl.metadata (3.5 kB)
Collecting pluggy<2,>=1.5 (from pytest>=8.0->warehouse-lab6==1.0.0)
  Downloading pluggy-1.7.0-py3-none-any.whl.metadata (5.0 kB)
Collecting pygments>=2.7.2 (from pytest>=8.0->warehouse-lab6==1.0.0)
  Using cached pygments-2.21.0-py3-none-any.whl.metadata (2.5 kB)
```

```
Building wheels for collected packages: warehouse-lab6
  Building editable for warehouse-lab6 (pyproject.toml) ... done
  Created wheel for warehouse-lab6: filename=warehouse_lab6-1.0.0-0.editable-py3-none-any.whl size=1393 sha256=ac9a
  Stored in directory: C:\Users\Анастасія\AppData\Local\Temp\pip-ephem-wheel-cache-kdptt5aj\wheels\d0\97\66\747ef14
Successfully built warehouse-lab6
Installing collected packages: warehouse-lab6, pygments, pluggy, packaging, iniconfig, coverage, colorama, pytest,
Successfully installed colorama-0.4.6 coverage-7.16.2 iniconfig-2.3.1 packaging-26.3 pluggy-1.7.0 pygments-2.21.0 p
1.0.0

[notice] A new release of pip is available: 26.1.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Запуск тестів:

```
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab6sveleba> python -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.13.14, pytest-9.1.1, pluggy-1.7.0 -- C:\Users\Анастасія\PycharmProjects\lab6sveleba\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Анастасія\PycharmProjects\lab6sveleba
configfile: pyproject.toml
testpaths: tests
plugins: asyncio-1.4.0, cov-7.1.0
asyncio: mode=Mode.AUTO, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 73 items

tests/integration/test_stock_flow.py::test_complete_stock_flow PASSED        [  1%]
tests/integration/test_stock_flow.py::test_csv_import_to_warehouse_and_json_export PASSED [  2%]
tests/integration/test_stock_flow.py::test_low_stock_reorder_workflow PASSED [  4%]
tests/unit/test_config.py::test_default_low_stock_threshold PASSED           [  5%]
tests/unit/test_config.py::test_custom_low_stock_threshold PASSED            [  6%]
tests/unit/test_config.py::test_invalid_low_stock_threshold PASSED           [  8%]
tests/unit/test_config.py::test_negative_low_stock_threshold PASSED          [  9%]
tests/unit/test_import_export.py::test_csv_import_from_file_object PASSED    [ 10%]
tests/unit/test_import_export.py::test_csv_import_streams_records PASSED     [ 12%]
tests/unit/test_import_export.py::test_csv_import_from_path PASSED           [ 13%]
tests/unit/test_import_export.py::test_csv_import_rejects_missing_columns PASSED [ 15%]
tests/unit/test_import_export.py::test_csv_import_rejects_invalid_quantity PASSED [ 16%]
tests/unit/test_import_export.py::test_csv_import_rejects_negative_quantity PASSED [ 17%]
tests/unit/test_import_export.py::test_csv_import_rejects_invalid_price PASSED [ 19%]
tests/unit/test_import_export.py::test_csv_import_rejects_empty_file PASSED  [ 20%]
tests/unit/test_import_export.py::test_json_export_to_path PASSED            [ 21%]
```

```
tests/unit/test_import_export.py::test_json_export_to_file_object PASSED     [ 23%]
tests/unit/test_import_export.py::test_json_export_empty_list PASSED         [ 24%]
tests/unit/test_models.py::test_product_creation PASSED                      [ 26%]
tests/unit/test_models.py::test_product_accepts_zero_price PASSED            [ 27%]
tests/unit/test_models.py::test_product_rejects_empty_code[] PASSED          [ 28%]
tests/unit/test_models.py::test_product_rejects_empty_code[ ] PASSED         [ 30%]
tests/unit/test_models.py::test_product_rejects_empty_code[\t] PASSED        [ 31%]
tests/unit/test_models.py::test_product_rejects_empty_name[] PASSED          [ 32%]
tests/unit/test_models.py::test_product_rejects_empty_name[ ] PASSED         [ 34%]
tests/unit/test_models.py::test_product_rejects_empty_name[\n] PASSED        [ 35%]
tests/unit/test_models.py::test_product_rejects_invalid_price[-1] PASSED     [ 36%]
tests/unit/test_models.py::test_product_rejects_invalid_price[-0.01] PASSED  [ 38%]
tests/unit/test_models.py::test_product_rejects_invalid_price[-inf] PASSED   [ 39%]
tests/unit/test_models.py::test_product_rejects_invalid_price[inf] PASSED    [ 41%]
tests/unit/test_models.py::test_product_rejects_invalid_price[nan] PASSED    [ 42%]
tests/unit/test_models.py::test_product_rejects_non_numeric_price[100] PASSED [ 43%]
tests/unit/test_models.py::test_product_rejects_non_numeric_price[None] PASSED [ 45%]
tests/unit/test_models.py::test_product_rejects_non_numeric_price[True] PASSED [ 46%]
tests/unit/test_models.py::test_warehouse_item_creation PASSED               [ 47%]
tests/unit/test_models.py::test_warehouse_item_rejects_negative_quantity[-1] PASSED [ 49%]
tests/unit/test_models.py::test_warehouse_item_rejects_negative_quantity[-10] PASSED [ 50%]
tests/unit/test_models.py::test_warehouse_item_rejects_non_integer_quantity[1.5] PASSED [ 52%]
tests/unit/test_models.py::test_warehouse_item_rejects_non_integer_quantity[5] PASSED [ 53%]
tests/unit/test_models.py::test_warehouse_item_rejects_non_integer_quantity[True] PASSED [ 54%]
tests/unit/test_models.py::test_warehouse_item_rejects_non_integer_quantity[None] PASSED [ 56%]
tests/unit/test_models.py::test_stock_value_uses_quantity_and_unit_price PASSED [ 57%]
```

```
tests/unit/test_models.py::test_receive_increases_quantity PASSED            [ 58%]
tests/unit/test_models.py::test_issue_decreases_quantity PASSED              [ 60%]
tests/unit/test_models.py::test_issue_exact_available_quantity PASSED        [ 61%]
tests/unit/test_models.py::test_issue_more_than_available_raises PASSED      [ 63%]
tests/unit/test_models.py::test_receive_rejects_non_positive_quantity[0] PASSED [ 64%]
tests/unit/test_models.py::test_receive_rejects_non_positive_quantity[-1] PASSED [ 65%]
tests/unit/test_models.py::test_receive_rejects_non_positive_quantity[-5] PASSED [ 67%]
tests/unit/test_models.py::test_issue_rejects_invalid_quantity[0] PASSED     [ 68%]
tests/unit/test_models.py::test_issue_rejects_invalid_quantity[-1] PASSED    [ 69%]
tests/unit/test_models.py::test_issue_rejects_invalid_quantity[1.2] PASSED   [ 71%]
tests/unit/test_models.py::test_issue_rejects_invalid_quantity[True] PASSED  [ 72%]
tests/unit/test_models.py::test_issue_rejects_invalid_quantity[2] PASSED     [ 73%]
tests/unit/test_services.py::test_repository_add_and_get PASSED              [ 75%]
tests/unit/test_services.py::test_repository_rejects_duplicate_code PASSED   [ 76%]
tests/unit/test_services.py::test_repository_all_returns_all_items PASSED    [ 78%]
tests/unit/test_services.py::test_register_product PASSED                    [ 79%]
tests/unit/test_services.py::test_find_product_returns_none_for_unknown_code PASSED [ 80%]
tests/unit/test_services.py::test_receive_updates_stock PASSED               [ 82%]
tests/unit/test_services.py::test_issue_updates_stock PASSED                 [ 83%]
tests/unit/test_services.py::test_receive_unknown_product_raises PASSED      [ 84%]
tests/unit/test_services.py::test_issue_unknown_product_raises PASSED        [ 86%]
tests/unit/test_services.py::test_service_issue_insufficient_stock PASSED    [ 87%]
tests/unit/test_services.py::test_total_warehouse_value PASSED               [ 89%]
tests/unit/test_services.py::test_low_stock_uses_configured_threshold PASSED [ 90%]
tests/unit/test_services.py::test_repository_mock_return_value PASSED        [ 91%]
tests/unit/test_services.py::test_repository_mock_side_effect PASSED         [ 93%]
tests/unit/test_services.py::test_service_interacts_with_repository PASSED   [ 94%]
tests/unit/test_services.py::test_invalid_low_stock_threshold_is_rejected PASSED [ 95%]
tests/unit/test_services.py::test_reorder_low_stock_uses_async_mock PASSED   [ 97%]
tests/unit/test_services.py::test_reorder_side_effect_reports_supplier_failure PASSED [ 98%]
tests/unit/test_services.py::test_reorder_rejects_invalid_quantity PASSED    [100%]

============================== 73 passed in 0.23s ==============================
```

Генерація HTML-звіту покриття:

```
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab6sveleba> python -m pytest -v --cov=src/warehouse --cov-branch --cov-report=term-missing --cov-report=html
============================= test session starts =============================
platform win32 -- Python 3.13.14, pytest-9.1.1, pluggy-1.7.0 -- C:\Users\Анастасія\PycharmProjects\lab6sveleba\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Анастасія\PycharmProjects\lab6sveleba
configfile: pyproject.toml
testpaths: tests
plugins: asyncio-1.4.0, cov-7.1.0
asyncio: mode=Mode.AUTO, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 73 items

tests/integration/test_stock_flow.py::test_complete_stock_flow PASSED        [  1%]
tests/integration/test_stock_flow.py::test_csv_import_to_warehouse_and_json_export PASSED [  2%]
tests/integration/test_stock_flow.py::test_low_stock_reorder_workflow PASSED [  4%]
tests/unit/test_config.py::test_default_low_stock_threshold PASSED           [  5%]
tests/unit/test_config.py::test_custom_low_stock_threshold PASSED            [  6%]
tests/unit/test_config.py::test_invalid_low_stock_threshold PASSED           [  8%]
tests/unit/test_config.py::test_negative_low_stock_threshold PASSED          [  9%]
tests/unit/test_import_export.py::test_csv_import_from_file_object PASSED    [ 10%]
tests/unit/test_import_export.py::test_csv_import_streams_records PASSED     [ 12%]
tests/unit/test_import_export.py::test_csv_import_from_path PASSED           [ 13%]
tests/unit/test_import_export.py::test_csv_import_rejects_missing_columns PASSED [ 15%]
tests/unit/test_import_export.py::test_csv_import_rejects_invalid_quantity PASSED [ 16%]
tests/unit/test_import_export.py::test_csv_import_rejects_negative_quantity PASSED [ 17%]
tests/unit/test_import_export.py::test_csv_import_rejects_invalid_price PASSED [ 19%]
tests/unit/test_import_export.py::test_csv_import_rejects_empty_file PASSED  [ 20%]
tests/unit/test_import_export.py::test_json_export_to_path PASSED            [ 21%]
```

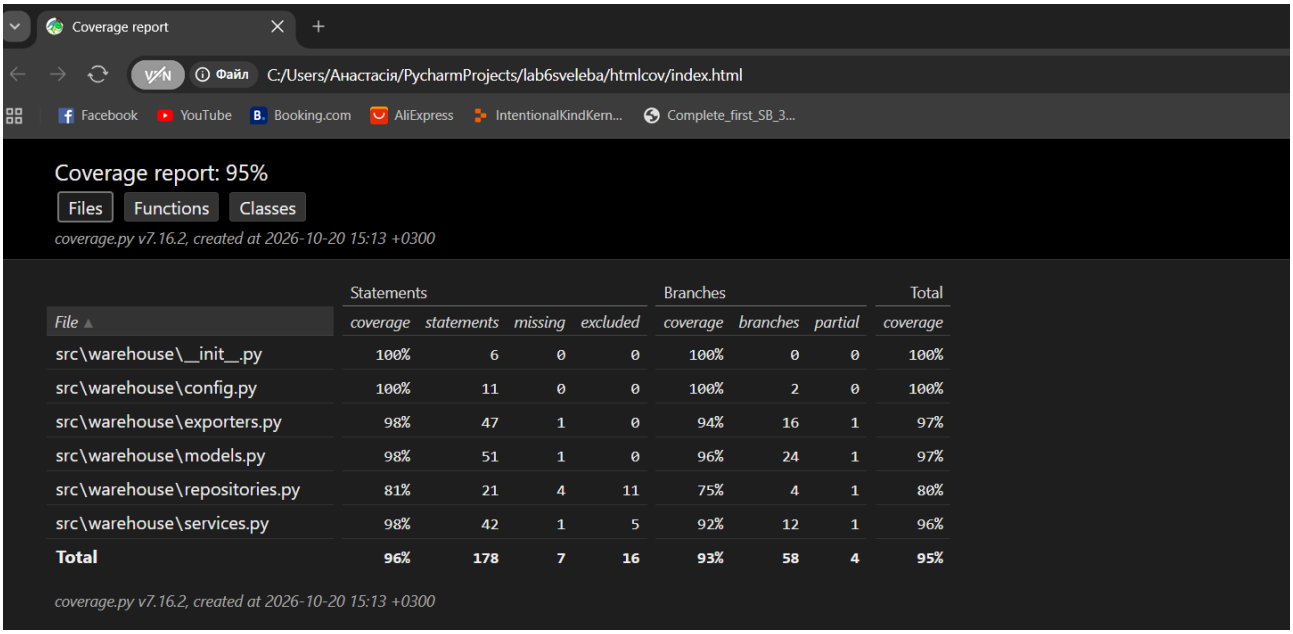
```
tests/unit/test_import_export.py::test_json_export_to_file_object PASSED     [ 23%]
tests/unit/test_import_export.py::test_json_export_empty_list PASSED         [ 24%]
tests/unit/test_models.py::test_product_creation PASSED                      [ 26%]
tests/unit/test_models.py::test_product_accepts_zero_price PASSED            [ 27%]
tests/unit/test_models.py::test_product_rejects_empty_code[] PASSED          [ 28%]
tests/unit/test_models.py::test_product_rejects_empty_code[ ] PASSED         [ 30%]
tests/unit/test_models.py::test_product_rejects_empty_code[\t] PASSED        [ 31%]
tests/unit/test_models.py::test_product_rejects_empty_name[] PASSED          [ 32%]
tests/unit/test_models.py::test_product_rejects_empty_name[ ] PASSED         [ 34%]
tests/unit/test_models.py::test_product_rejects_empty_name[\n] PASSED        [ 35%]
tests/unit/test_models.py::test_product_rejects_invalid_price[-1] PASSED     [ 36%]
tests/unit/test_models.py::test_product_rejects_invalid_price[-0.01] PASSED  [ 38%]
tests/unit/test_models.py::test_product_rejects_invalid_price[-inf] PASSED   [ 39%]
tests/unit/test_models.py::test_product_rejects_invalid_price[inf] PASSED    [ 41%]
tests/unit/test_models.py::test_product_rejects_invalid_price[nan] PASSED    [ 42%]
tests/unit/test_models.py::test_product_rejects_non_numeric_price[100] PASSED [ 43%]
tests/unit/test_models.py::test_product_rejects_non_numeric_price[None] PASSED [ 45%]
tests/unit/test_models.py::test_product_rejects_non_numeric_price[True] PASSED [ 46%]
tests/unit/test_models.py::test_warehouse_item_creation PASSED               [ 47%]
tests/unit/test_models.py::test_warehouse_item_rejects_negative_quantity[-1] PASSED [ 49%]
tests/unit/test_models.py::test_warehouse_item_rejects_negative_quantity[-10] PASSED [ 50%]
tests/unit/test_models.py::test_warehouse_item_rejects_non_integer_quantity[1.5] PASSED [ 52%]
tests/unit/test_models.py::test_warehouse_item_rejects_non_integer_quantity[5] PASSED [ 53%]
tests/unit/test_models.py::test_warehouse_item_rejects_non_integer_quantity[True] PASSED [ 54%]
tests/unit/test_models.py::test_warehouse_item_rejects_non_integer_quantity[None] PASSED [ 56%]
tests/unit/test_models.py::test_stock_value_uses_quantity_and_unit_price PASSED [ 57%]
tests/unit/test_models.py::test_receive_increases_quantity PASSED            [ 58%]
```

```
tests/unit/test_models.py::test_issue_decreases_quantity PASSED              [ 60%]
tests/unit/test_models.py::test_issue_exact_available_quantity PASSED        [ 61%]
tests/unit/test_models.py::test_issue_more_than_available_raises PASSED      [ 63%]
tests/unit/test_models.py::test_receive_rejects_non_positive_quantity[0] PASSED [ 64%]
tests/unit/test_models.py::test_receive_rejects_non_positive_quantity[-1] PASSED [ 65%]
tests/unit/test_models.py::test_receive_rejects_non_positive_quantity[-5] PASSED [ 67%]
tests/unit/test_models.py::test_issue_rejects_invalid_quantity[0] PASSED     [ 68%]
tests/unit/test_models.py::test_issue_rejects_invalid_quantity[-1] PASSED    [ 69%]
tests/unit/test_models.py::test_issue_rejects_invalid_quantity[1.2] PASSED   [ 71%]
tests/unit/test_models.py::test_issue_rejects_invalid_quantity[True] PASSED  [ 72%]
tests/unit/test_models.py::test_issue_rejects_invalid_quantity[2] PASSED     [ 73%]
tests/unit/test_services.py::test_repository_add_and_get PASSED              [ 75%]
tests/unit/test_services.py::test_repository_rejects_duplicate_code PASSED   [ 76%]
tests/unit/test_services.py::test_repository_all_returns_all_items PASSED    [ 78%]
tests/unit/test_services.py::test_register_product PASSED                    [ 79%]
tests/unit/test_services.py::test_find_product_returns_none_for_unknown_code PASSED [ 80%]
tests/unit/test_services.py::test_receive_updates_stock PASSED               [ 82%]
tests/unit/test_services.py::test_issue_updates_stock PASSED                 [ 83%]
tests/unit/test_services.py::test_receive_unknown_product_raises PASSED      [ 84%]
tests/unit/test_services.py::test_issue_unknown_product_raises PASSED        [ 86%]
tests/unit/test_services.py::test_service_issue_insufficient_stock PASSED    [ 87%]
tests/unit/test_services.py::test_total_warehouse_value PASSED               [ 89%]
tests/unit/test_services.py::test_low_stock_uses_configured_threshold PASSED [ 90%]
tests/unit/test_services.py::test_repository_mock_return_value PASSED        [ 91%]
tests/unit/test_services.py::test_repository_mock_side_effect PASSED         [ 93%]
tests/unit/test_services.py::test_service_interacts_with_repository PASSED   [ 94%]
tests/unit/test_services.py::test_invalid_low_stock_threshold_is_rejected PASSED [ 95%]
```

```
tests/unit/test_services.py::test_reorder_low_stock_uses_async_mock PASSED   [ 97%]
tests/unit/test_services.py::test_reorder_side_effect_reports_supplier_failure PASSED [ 98%]
tests/unit/test_services.py::test_reorder_rejects_invalid_quantity PASSED    [100%]

================================== tests coverage ==================================
_____________ coverage: platform win32, python 3.13.14-final-0 _____________

Name                           Stmts   Miss Branch BrPart  Cover   Missing
--------------------------------------------------------------------------
src\warehouse\__init__.py          6      0      0      0   100%
src\warehouse\config.py           11      0      2      0   100%
src\warehouse\exporters.py        47      1     16      1    97%   54
src\warehouse\models.py           51      1     24      1    97%   70
src\warehouse\repositories.py     21      4      4      1    80%   30, 51-53
src\warehouse\services.py         42      1     12      1    96%   27
--------------------------------------------------------------------------
TOTAL                            178      7     58      4    95%
Coverage HTML written to dir htmlcov
Required test coverage of 80.0% reached. Total coverage: 95.34%
============================== 73 passed in 0.41s ==============================
```



Coverage report: 95%

Files | Functions | Classes

coverage.py v7.16.2, created at 2026-10-20 15:13 +0300

	Statements				Branches			Total
File ▲	*coverage*	*statements*	*missing*	*excluded*	*coverage*	*branches*	*partial*	*coverage*
src\warehouse__init__.py	100%	6	0	0	100%	0	0	100%
src\warehouse\config.py	100%	11	0	0	100%	2	0	100%
src\warehouse\exporters.py	98%	47	1	0	94%	16	1	97%
src\warehouse\models.py	98%	51	1	0	96%	24	1	97%
src\warehouse\repositories.py	81%	21	4	11	75%	4	1	80%
src\warehouse\services.py	98%	42	1	5	92%	12	1	96%
Total	**96%**	**178**	**7**	**16**	**93%**	**58**	**4**	**95%**

coverage.py v7.16.2, created at 2026-10-20 15:13 +0300

Coverage report: 95%

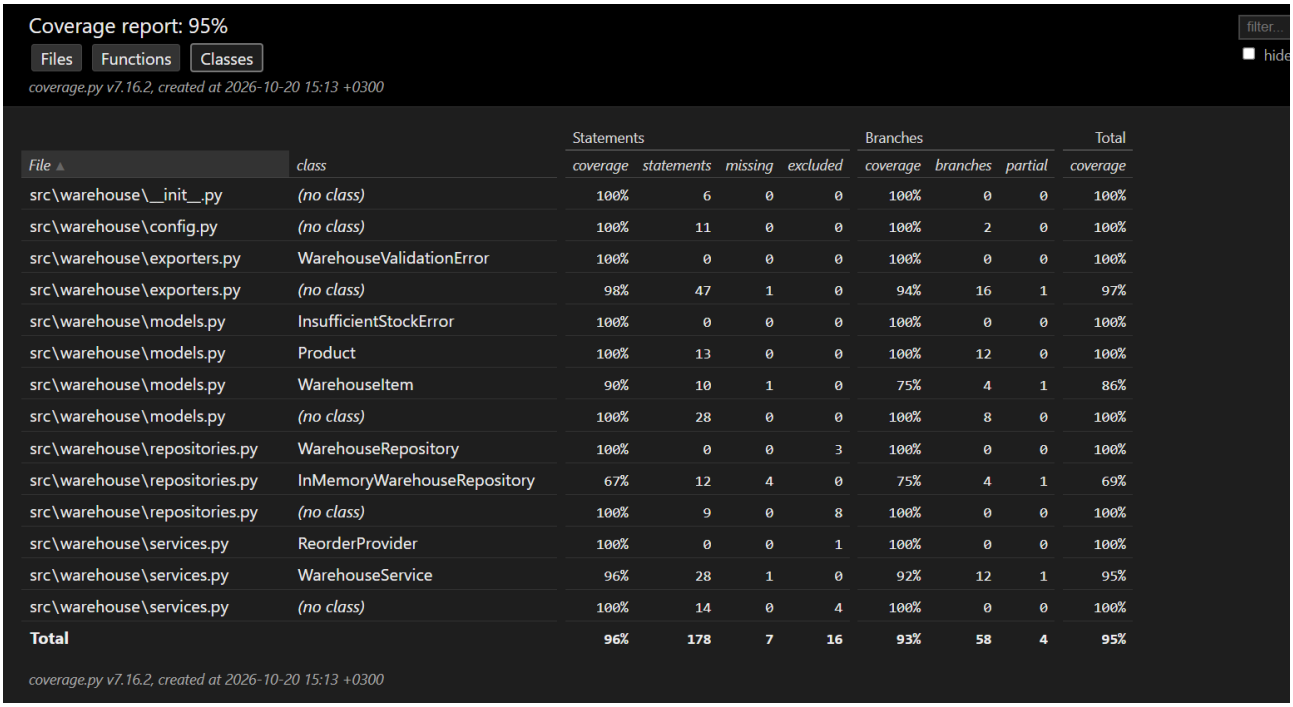
Files | Functions | Classes

filter... hide covere

coverage.py v7.16.2, created at 2026-10-20 15:13 +0300

		Statements				Branches			Total
File ▲	*function*	*coverage*	*statements*	*missing*	*excluded*	*coverage*	*branches*	*partial*	*coverage*
src\warehouse__init__.py	*(no function)*	100%	6	0	0	100%	0	0	100%
src\warehouse\config.py	get_low_stock_threshold	100%	8	0	0	100%	2	0	100%
src\warehouse\config.py	*(no function)*	100%	3	0	0	100%	0	0	100%
src\warehouse\exporters.py	stream_csv_import	97%	29	1	0	93%	14	1	95%
src\warehouse\exporters.py	export_json	100%	9	0	0	100%	2	0	100%
src\warehouse\exporters.py	*(no function)*	100%	9	0	0	100%	0	0	100%
src\warehouse\models.py	_validate_quantity	100%	4	0	0	100%	4	0	100%
src\warehouse\models.py	_validate_positive_quantity	100%	4	0	0	100%	4	0	100%
src\warehouse\models.py	Product.__post_init__	100%	13	0	0	100%	12	0	100%
src\warehouse\models.py	WarehouseItem.__post_init__	67%	3	1	0	50%	2	1	60%
src\warehouse\models.py	WarehouseItem.stock_value	100%	1	0	0	100%	0	0	100%
src\warehouse\models.py	WarehouseItem.receive	100%	2	0	0	100%	0	0	100%
src\warehouse\models.py	WarehouseItem.issue	100%	4	0	0	100%	2	0	100%
src\warehouse\models.py	*(no function)*	100%	20	0	0	100%	0	0	100%
src\warehouse\repositories.py	WarehouseRepository.add	100%	0	0	1	100%	0	0	100%
src\warehouse\repositories.py	WarehouseRepository.get	100%	0	0	1	100%	0	0	100%
src\warehouse\repositories.py	WarehouseRepository.all	100%	0	0	1	100%	0	0	100%
src\warehouse\repositories.py	InMemoryWarehouseRepository.__init__	100%	1	0	0	100%	0	0	100%
src\warehouse\repositories.py	InMemoryWarehouseRepository.add	83%	6	1	0	75%	4	1	80%
src\warehouse\repositories.py	InMemoryWarehouseRepository.get	100%	1	0	0	100%	0	0	100%
src\warehouse\repositories.py	InMemoryWarehouseRepository.all	100%	1	0	0	100%	0	0	100%
src\warehouse\repositories.py	InMemoryWarehouseRepository.register_product	0%	3	3	0	100%	0	0	0%
src\warehouse\repositories.py	*(no function)*	100%	9	0	8	100%	0	0	100%
src\warehouse\services.py	ReorderProvider.reorder	100%	0	0	1	100%	0	0	100%
src\warehouse\services.py	WarehouseService.__init__	83%	6	1	0	75%	4	1	80%
src\warehouse\services.py	WarehouseService.register_product	100%	3	0	0	100%	0	0	100%
src\warehouse\services.py	WarehouseService.find_product	100%	1	0	0	100%	0	0	100%
src\warehouse\services.py	WarehouseService.receive	100%	5	0	0	100%	2	0	100%
src\warehouse\services.py	WarehouseService.issue	100%	5	0	0	100%	2	0	100%
src\warehouse\services.py	WarehouseService.total_value	100%	1	0	0	100%	0	0	100%
src\warehouse\services.py	WarehouseService.low_stock_items	100%	1	0	0	100%	0	0	100%
src\warehouse\services.py	WarehouseService.reorder_low_stock	100%	6	0	0	100%	4	0	100%
src\warehouse\services.py	*(no function)*	100%	14	0	4	100%	0	0	100%
Total		**96%**	**178**	**7**	**16**	**93%**	**58**	**4**	**95%**

coverage.py v7.16.2, created at 2026-10-20 15:13 +0300



Coverage report: 95%

Files | Functions | Classes

coverage.py v7.16.2, created at 2026-10-20 15:13 +0300

filter... hide

		Statements				Branches			Total
File ▲	class	coverage	statements	missing	excluded	coverage	branches	partial	coverage
src\warehouse__init__.py	(no class)	100%	6	0	0	100%	0	0	100%
src\warehouse\config.py	(no class)	100%	11	0	0	100%	2	0	100%
src\warehouse\exporters.py	WarehouseValidationError	100%	0	0	0	100%	0	0	100%
src\warehouse\exporters.py	(no class)	98%	47	1	0	94%	16	1	97%
src\warehouse\models.py	InsufficientStockError	100%	0	0	0	100%	0	0	100%
src\warehouse\models.py	Product	100%	13	0	0	100%	12	0	100%
src\warehouse\models.py	WarehouseItem	90%	10	1	0	75%	4	1	86%
src\warehouse\models.py	(no class)	100%	28	0	0	100%	8	0	100%
src\warehouse\repositories.py	WarehouseRepository	100%	0	0	3	100%	0	0	100%
src\warehouse\repositories.py	InMemoryWarehouseRepository	67%	12	4	0	75%	4	1	69%
src\warehouse\repositories.py	(no class)	100%	9	0	8	100%	0	0	100%
src\warehouse\services.py	ReorderProvider	100%	0	0	1	100%	0	0	100%
src\warehouse\services.py	WarehouseService	96%	28	1	0	92%	12	1	95%
src\warehouse\services.py	(no class)	100%	14	0	4	100%	0	0	100%
Total		**96%**	**178**	**7**	**16**	**93%**	**58**	**4**	**95%**

coverage.py v7.16.2, created at 2026-10-20 15:13 +0300

Висновок: у ході виконання лабораторної роботи я розробила систему керування складськими запасами мовою Python, яка забезпечує реєстрацію товарів, облік їхньої кількості, надходження та списання продукції, обчислення загальної вартості запасів і визначення товарів із низьким залишком. Також я реалізувала імпорт даних із CSV-файлів, експорт інформації у формат JSON, налаштування параметрів системи через змінні середовища та асинхронну взаємодію з постачальником. Для перевірки правильності роботи програми було створено модульні й інтеграційні тести з використанням pytest, фікстур, mock-об’єктів і засобів тестування асинхронного коду. У процесі тестування перевірено коректність роботи основних компонентів системи, обробку некоректних даних і виникнення помилкових ситуацій. У результаті виконання роботи я отримала практичні навички організації Python-проєкту, написання автоматизованих тестів, перевірки взаємодії між компонентами та налаштування контролю покриття коду тестами.